

docker

Test de repaso — parte 3: Docker Compose

100 preguntas · 4 opciones, 1 correcta

Cómo usar este banco de preguntas

- Cada diapositiva contiene una pregunta con 4 opciones (A-D), solo 1 correcta.
- No incluye las respuestas: úsalo para practicar de cara al cuestionario. TODAS las preguntas del cuestionario saldrán de este banco de preguntas.
- Este cuestionario cierra el bloque 1.2 Docker: repasa también conceptos de las semanas 1 y 2 cuando se combinen con Compose.

Bloque 1

Por qué Docker Compose

Pregunta 1

Trabajas en 3 proyectos que necesitan `mariadb`, `redis`, `mongo`, `nginx` y `php-fpm`, cada uno con una imagen para desarrollo y otra para producción. ¿Por qué este escenario (unas 30 imágenes con parámetros de arranque distintos) ilustra bien la necesidad de Compose?

- A) Porque Docker no permite tener más de 10 imágenes descargadas a la vez
- B) Porque arrancar cada contenedor a mano con `docker run`, recordando todos sus parámetros, se vuelve inviable de gestionar según crece el número de servicios
- C) Porque Compose reduce automáticamente el número de imágenes necesarias a una sola
- D) Porque sin Compose no se podría tener dev y prod en el mismo equipo

Pregunta 2

Quieres levantar un WordPress con Compose sin escribir el `docker-compose.yml` completamente desde cero. ¿Qué es razonable hacer primero?

- A) Escribirlo a ciegas y depurar los errores uno a uno sin consultar nada
- B) Partir de un ejemplo oficial mantenido, como el de WordPress en el repositorio `docker/awesome-compose` de GitHub, y adaptarlo
- C) Pedir la contraseña de root de WordPress a Docker Hub
- D) Usar `docker compose init --wordpress`, que genera el fichero automáticamente

Pregunta 3

Necesitas consultar la lista completa y oficial de todas las claves que admite un fichero Compose (incluidas las menos usadas, como `profiles` o `deploy`). ¿Dónde la consultarías?

- A) En la wiki interna de Docker Hub, apartado 'Compose avanzado'
- B) En la referencia oficial del fichero Compose en docs.docker.com (reference/compose-file)
- C) Solo en el código fuente de Docker Compose, no hay documentación pública
- D) En el `man docker-compose` del sistema, que siempre está actualizado

Pregunta 4

¿Qué problema resuelve principalmente Docker Compose?

- A) Sustituir completamente a Docker como motor de contenedores
- B) Definir y levantar, con un solo comando, una aplicación formada por varios contenedores que trabajan juntos
- C) Sustituir a git como sistema de control de versiones
- D) Comprimir el tamaño de las imágenes Docker ya construidas

Pregunta 5

Sin Docker Compose, si una aplicación necesita una app web y una base de datos en contenedores separados que se comuniquen entre sí, ¿qué habría que hacer?

- A) Nada especial: Docker los conecta automáticamente aunque se usen `docker run` sueltos
- B) Ejecutar varios `docker run` por separado, creando a mano la red y coordinando el orden de arranque
- C) Sería imposible tener dos contenedores funcionando a la vez en la misma máquina
- D) Escribir un único Dockerfile que construya ambos contenedores en una sola imagen

Pregunta 6

¿Qué ventaja aporta versionar en git un `docker-compose.yml` junto con el código de un proyecto?

- A) Ninguna relevante: el entorno se puede reconstruir igual sin documentarlo
- B) Cualquier miembro del equipo puede levantar exactamente el mismo entorno completo con un solo comando, de forma reproducible
- C) Hace innecesario seguir usando Dockerfiles en el proyecto
- D) Evita tener que instalar Docker en las máquinas del equipo

Pregunta 7

¿Es correcto decir que Docker Compose sustituye por completo al Dockerfile?

- A) Sí, a partir de Compose ya no hace falta ningún Dockerfile en el proyecto
- B) No: el Dockerfile sigue definiendo cómo se construye una imagen; Compose define cómo se levantan y relacionan varios servicios, que pueden usar esas imágenes
- C) Sí, Compose genera automáticamente las imágenes sin necesidad de instrucciones
- D) No existe relación alguna entre ambos conceptos

Pregunta 8

¿Puede Docker Compose usarse de forma razonable para levantar una aplicación con un único servicio?

- A) No, Compose exige un mínimo de dos servicios para funcionar
- B) Sí, aunque su mayor utilidad se nota cuando hay varios servicios relacionados entre sí
- C) No, Compose solo admite servicios de base de datos
- D) Solo si ese servicio no usa ninguna imagen de Docker Hub

Pregunta 9

En versiones recientes de Docker, ¿cuál es la forma recomendada de invocar Compose, frente al antiguo binario independiente `docker-compose` ?

- A) `docker compose` , como subcomando integrado del propio CLI de Docker
- B) `docker-run-compose` , el binario que sustituyó a `docker-compose`
- C) `dockercompose` , sin espacio ni guion
- D) `compose docker` , invirtiendo el orden de las palabras

Bloque 2

Estructura de docker-compose.yml

Pregunta 10

¿Cuál es la clave de primer nivel donde normalmente se define cada contenedor de la aplicación en un `docker-compose.yml` ?

- A) `containers:`
- B) `services:`
- C) `apps:`
- D) `dockers:`

Pregunta 11

En YAML, ¿qué determina principalmente la jerarquía (qué pertenece a qué)?

- A) El uso de llaves `{}` como en JSON
- B) La indentación con espacios (nunca tabuladores)
- C) El uso de paréntesis
- D) Que cada clave vaya entre comillas dobles

Pregunta 12

Un alumno indenta su `docker-compose.yml` mezclando tabuladores y espacios. ¿Qué es lo más probable que ocurra al ejecutar `docker compose up`?

- A) Funciona igual, YAML normaliza tabuladores y espacios automáticamente
- B) Compose devuelve un error de parseo del YAML y no levanta ningún servicio
- C) Solo se ignoran las líneas con tabuladores, el resto funciona
- D) Docker convierte el fichero a JSON automáticamente para evitar el problema

Pregunta 13

Dado este fragmento:

```
services:  
  web:  
    image: nginx  
  db:  
    image: postgres
```

¿Cuántos servicios define?

- A) 1
- B) 2
- C) 3
- D) 0, falta la clave `containers:`

Pregunta 14

¿Qué otras claves de primer nivel, además de `services:`, son habituales en un `docker-compose.yml`?

- A) `volumes:` y `networks:`, para declarar volúmenes y redes compartidos entre servicios
- B) `dockerfile:` y `build.py:`
- C) `teoria:` y `practicas:`
- D) Ninguna más: `services:` es la única clave que admite el formato

Pregunta 15

Tienes `ports`:

- `"8080:80"` en un servicio. ¿Por qué suele escribirse entre comillas el valor `8080:80`?

- A) Porque YAML lo exige siempre para cualquier texto, sin excepción
- B) Para evitar que el `:` se interprete como separador de clave-valor de YAML en vez de como parte del valor
- C) Porque sin comillas Docker no reconoce los puertos
- D) Las comillas no tienen ningún efecto, es solo una convención estética

Pregunta 16

Si en `environment:` usas `${DB_PASSWORD}` pero esa variable no está definida ni en `.env` ni en el entorno, ¿qué es lo más probable que ocurra?

- A) Docker Compose aborta con un error fatal sí o sí
- B) Compose sustituye la variable por una cadena vacía (o avisa), en vez de bloquear el arranque del servicio
- C) Se usa automáticamente la contraseña por defecto de la imagen
- D) El fichero deja de ser YAML válido

Pregunta 17

¿Qué comando de Compose permite validar y ver cómo queda interpretado un `docker-compose.yml` (con variables ya sustituidas) sin llegar a arrancar nada?

- A) `docker compose config`
- B) `docker compose validate-only`
- C) `docker compose dry-run`
- D) `docker compose lint`

Pregunta 18

¿Puede un `docker-compose.yml` referenciar un `Dockerfile` propio en vez de usar siempre una imagen ya publicada?

- A) No, Compose solo admite imágenes descargadas de Docker Hub
- B) Sí, mediante la clave `build:` apuntando a la carpeta donde está el Dockerfile
- C) No, Compose nunca construye imágenes, solo las ejecuta
- D) Solo si el Dockerfile se llama exactamente `compose.Dockerfile`

Bloque 3

services: image y build

Pregunta 19

Dentro de un servicio, ¿qué indica la clave `image:` ?

- A) El nombre que tendrá el contenedor en ejecución
- B) Qué imagen Docker (ya existente, p. ej. en Docker Hub) usar para ese servicio
- C) La carpeta del proyecto donde vive el código
- D) El puerto que se va a publicar

Pregunta 20

¿Se pueden combinar `build:` e `image:` en el mismo servicio?

- A) No, son mutuamente excluyentes en cualquier caso
- B) Sí: Compose construye la imagen con `build:` y la etiqueta con el nombre indicado en `image:`
- C) `image:` siempre tiene prioridad e ignora por completo `build:`
- D) Solo si el servicio no pertenece a ningún `services:`

Pregunta 21

Dado:

```
services:  
  app:  
    build: ./app
```

¿Qué se espera encontrar dentro de la carpeta `./app` para que el build funcione?

- A) Un `Dockerfile` (y el código fuente necesario) para construir la imagen de ese servicio
- B) Una imagen ya construida en formato `.tar`
- C) Un `docker-compose.yml` adicional obligatorio
- D) Nada: la carpeta solo se usa como nombre, no como contenido

Pregunta 22

¿Qué ventaja aporta usar `image: postgres:16` en vez de construirte tu propia imagen de base de datos con `build: ?`

- A) Ninguna: siempre es preferible construir tu propia imagen de base de datos
- B) Aprovechas una imagen oficial ya probada y mantenida, sin tener que escribir ni mantener tú mismo un Dockerfile para ello
- C) `image:` no es compatible con bases de datos
- D) `build:` es obligatorio para cualquier servicio con estado

Pregunta 23

¿Qué clave permite asignar un nombre de contenedor fijo a un servicio, en vez del generado automáticamente por Compose?

- A) `container_name:`
- B) `service_name:`
- C) `name:`
- D) `hostname_container:`

Pregunta 24

¿Qué ocurre si un servicio de un `docker-compose.yml` no define ni `image:` ni `build:` ?

- A) Compose asigna automáticamente la imagen `ubuntu` por defecto
- B) Compose da un error de configuración, porque no sabe qué imagen usar para levantar ese servicio
- C) El servicio se ejecuta en un modo simulado sin crear ningún contenedor real
- D) Usa la imagen más reciente que haya en el sistema, sea cual sea

Pregunta 25

Un alumno escribe `build: ./app` pero el Dockerfile de esa carpeta se llama `Dockerfile.dev` (no `Dockerfile` a secas). Al ejecutar `docker compose up --build`, ¿qué es lo más probable que pase?

- A) Compose detecta automáticamente cualquier nombre de Dockerfile sin configuración extra
- B) Falla al no encontrar un fichero llamado `Dockerfile`, salvo que se indique explícitamente con `dockerfile:` dentro de `build:`
- C) Construye la imagen usando el último Dockerfile usado en cualquier proyecto
- D) Ignora el build y usa `image: app:latest` por defecto

Pregunta 26

¿Para qué sirve la clave `command:` dentro de un servicio?

- A) Para documentar en qué red está el servicio
- B) Para sobrescribir el comando por defecto (`CMD`) que traía la imagen
- C) Para declarar variables de entorno
- D) Para fijar el puerto publicado

Pregunta 27

Dos servicios distintos necesitan construirse desde el mismo Dockerfile pero con distinta configuración en tiempo de ejecución. ¿Qué clave usarías para diferenciarlos sin duplicar el Dockerfile?

- A) `environment:` , pasando variables distintas a cada servicio
- B) `networks:` distintas obligatoriamente
- C) `ports:` distintos únicamente
- D) No es posible sin duplicar el Dockerfile

Bloque 4

ports, environment y secretos

Pregunta 28

¿Qué clave de un servicio equivale, en Compose, a la opción `-p` de `docker run` ?

- A) `ports:`
- B) `expose:`
- C) `network:`
- D) `volumes:`

Pregunta 29

Dado:

```
services:  
  web:  
    image: nginx  
    ports:  
      - "8080:80"
```

¿Qué puerto usarías desde tu navegador en el host para acceder a la app?

- A) 80
- B) 8080
- C) Cualquiera de los dos, da igual
- D) Ninguno, faltaría `expose:`

Pregunta 30

Tu aplicación, dentro del contenedor, escucha en el puerto 3000, pero en el `docker-compose.yml` has puesto `ports: ["8080:5000"]`. ¿Qué es lo más probable que pase al intentar acceder a `localhost:8080` ?

- A) Funciona perfectamente, Compose ajusta el puerto interno automáticamente
- B) No responde correctamente, porque el puerto del contenedor indicado (5000) no coincide con el puerto real donde escucha la app (3000)
- C) Compose avisa y corrige el error antes de arrancar
- D) Se publica automáticamente también el puerto 3000

Pregunta 31

¿Cuál de estas dos formas de declarar `environment:` es válida en Compose?

- A) Solo como lista (`- VAR=valor`)
- B) Tanto como lista (`- VAR=valor`) como como diccionario (`VAR: valor`)
- C) Solo como diccionario
- D) Ninguna: siempre hay que usar un fichero `.env`

Pregunta 32

¿Para qué sirve un fichero `.env` situado junto al `docker-compose.yml` ?

- A) Para nada: Compose lo ignora por completo
- B) Para definir variables que Compose sustituye automáticamente dentro del propio YAML (p. ej. `${DB_PASSWORD}`)
- C) Es obligatorio y sustituye siempre a `environment:`
- D) Solo tiene efecto en producción, nunca en desarrollo

Pregunta 33

Dos servicios del mismo `docker-compose.yml` publican `ports: ["8080:80"]` cada uno. ¿Qué ocurre al intentar levantar ambos a la vez?

- A) Compose reparte automáticamente otro puerto libre al segundo
- B) Se produce un conflicto porque el puerto 8080 del host ya estaría en uso
- C) No pasa nada, ambos funcionan igual de forma simultánea
- D) Solo se publica el segundo servicio, el primero se ignora en silencio

Pregunta 34

¿Es obligatorio declarar `ports:` para que dos servicios de la misma Compose se comuniquen entre sí?

- A) Sí, siempre
- B) No: los servicios de un mismo proyecto Compose pueden comunicarse internamente por su nombre, sin publicar puertos al host
- C) Solo si están en ficheros Compose distintos
- D) No es posible que se comuniquen sin publicar puertos

Pregunta 35

Quieres que una base de datos solo sea accesible desde otros servicios de la misma Compose, nunca desde el navegador/host. ¿Qué harías?

- A) No declarar `ports:` para ese servicio, dejando que se use solo por la red interna
- B) Publicar igualmente el puerto, no hay otra forma de limitarlo
- C) Sustituir `ports:` por `environment:`
- D) Eliminar el servicio de base de datos por completo

Pregunta 36

Un equipo sube por error su `.env` de producción (con contraseñas reales) a un repositorio público que usa Compose. ¿Qué riesgo concreto implica esto?

- A) Ninguno: `.env` nunca llega a subirse de verdad
- B) Cualquiera con acceso al repositorio podría leer y usar esas credenciales
- C) Docker cifra automáticamente el contenido de `.env`
- D) Compose elimina las contraseñas del `.env` al hacer `up`

Bloque 5

volumes en Compose

Pregunta 37

¿Para qué se usa principalmente la clave `volumes:` dentro de un servicio?

- A) Para declarar variables de entorno
- B) Para montar datos persistentes o compartir ficheros entre el host (o un volumen gestionado) y el contenedor
- C) Para publicar puertos del servicio
- D) Para fijar el nombre del contenedor

Pregunta 38

Dado:

```
services:
  db:
    image: postgres
    volumes:
      - db_data:/var/lib/postgresql/data
volumes:
  db_data:
```

¿Qué es `db_data` ?

- A) Una carpeta del host con exactamente esa ruta
- B) Un volumen con nombre, gestionado por Docker, declarado en la sección `volumes:` de nivel superior
- C) Una variable de entorno del servicio `db`
- D) El nombre de la imagen usada

Pregunta 39

¿Por qué es importante usar un volumen para los datos de un servicio de base de datos en Compose?

- A) Para que la base de datos ocupe menos espacio en disco
- B) Para que los datos sobrevivan aunque se elimine y se vuelva a crear el contenedor (p. ej. con `docker compose down`)
- C) No es importante: los datos de un contenedor nunca se pierden
- D) Solo mejora el rendimiento de red, no afecta a la persistencia

Pregunta 40

¿Qué diferencia hay entre un bind mount (`./datos:/var/lib/data`) y un volumen con nombre (`db_data:/var/lib/data`)?

- A) Ninguna: ambos son exactamente equivalentes
- B) El bind mount usa una ruta concreta del proyecto en el host; el volumen con nombre lo crea y administra Docker internamente
- C) El volumen con nombre no puede usarse con bases de datos
- D) El bind mount no permite nunca escritura desde el contenedor

Pregunta 41

Si ejecutas `docker compose down` sin ninguna opción adicional, ¿qué ocurre con los volúmenes con nombre del proyecto?

- A) Se eliminan siempre, junto con los contenedores
- B) Por defecto se conservan: hace falta indicarlo explícitamente (p. ej. con `-v`) para eliminarlos
- C) Se convierten automáticamente en imágenes
- D) Se mueven a Docker Hub

Pregunta 42

¿Qué opción de `docker compose down` elimina también los volúmenes con nombre asociados al proyecto?

- A) `--volumes` o `-v`
- B) `--force`
- C) `--hard`
- D) `--clean`

Pregunta 43

¿Puede un mismo volumen con nombre compartirse entre varios servicios de un mismo `docker-compose.yml` ?

- A) No, cada volumen solo lo puede montar un único servicio
- B) Sí, varios servicios pueden montar el mismo volumen para compartir datos entre ellos
- C) Solo si ambos servicios usan la misma imagen
- D) Solo si están en redes distintas

Pregunta 44

En desarrollo, es habitual montar el código fuente como bind mount dentro del contenedor (p. ej. `. : /app`) en vez de solo copiarlo con `COPY` en el Dockerfile. ¿Qué ventaja práctica aporta esto durante el desarrollo?

- A) Ninguna: siempre es mejor reconstruir la imagen tras cada cambio
- B) Los cambios hechos en el código del host se reflejan al instante dentro del contenedor, sin tener que reconstruir la imagen
- C) Hace que el contenedor sea automáticamente más seguro
- D) Es un requisito obligatorio para que Compose funcione

Pregunta 45

¿Qué comando lista los volúmenes existentes en Docker, incluidos los creados por Compose?

- A) `docker volume ls`
- B) `docker compose volumes`
- C) `docker ps --volumes`
- D) `docker images --volumes`

Bloque 6

networks en Compose

Pregunta 46

Por defecto, al ejecutar `docker compose up`, ¿qué hace Compose respecto a las redes?

- A) No crea ninguna red: todos los servicios usan la red del host directamente
- B) Crea automáticamente una red propia para ese proyecto y conecta a ella todos sus servicios
- C) Exige que el usuario cree la red a mano antes de arrancar
- D) Usa siempre la red `host` del sistema

Pregunta 47

`web` y `db` están definidos en el mismo `docker-compose.yml`, usando la red por defecto del proyecto. ¿Cómo puede `web` conectarse a `db`?

- A) A través de la IP pública de internet de `db`
- B) Usando el nombre del servicio `db` como si fuera un hostname, en la cadena de conexión
- C) No es posible que se comuniquen sin publicar antes el puerto de `db`
- D) Solo mediante `localhost`, igual que si estuvieran en la misma máquina física

Pregunta 48

¿Es correcto usar `localhost` dentro del código de un servicio para referirse a otro servicio de la misma Compose?

- A) Sí, siempre funciona igual que el nombre del servicio
- B) No: `localhost` dentro de un contenedor se refiere a sí mismo, no a otros servicios; hay que usar el nombre del servicio
- C) Solo en sistemas Windows
- D) Solo si no se ha declarado ninguna red personalizada

Pregunta 49

¿Para qué se usaría la clave `networks:` de nivel superior (fuera de cualquier servicio) en un `docker-compose.yml` ?

- A) Para declarar redes personalizadas adicionales a la red por defecto del proyecto
- B) Para definir variables de entorno compartidas
- C) Para construir las imágenes de los servicios
- D) Esa clave no existe en Compose

Pregunta 50

¿Qué ventaja aporta separar los servicios de una Compose en varias redes (p. ej. una "frontend" y otra "backend")?

- A) Ninguna: siempre es mejor usar una única red para todo
- B) Permite aislar qué servicios pueden comunicarse entre sí, por ejemplo que la base de datos solo sea visible desde la red "backend"
- C) Hace que la aplicación consuma menos CPU
- D) Es obligatorio declarar varias redes en cualquier Compose

Pregunta 51

Si un servicio no se declara explícitamente en ninguna red personalizada dentro de una Compose que sí las define, ¿a qué red pertenece por defecto?

- A) A ninguna: queda completamente aislado
- B) A la red por defecto del proyecto, si no se le indica otra cosa
- C) Da un error de configuración obligatoriamente
- D) Se conecta directamente a internet sin pasar por el motor de Docker

Pregunta 52

¿Qué comando permite ver las redes existentes en Docker, incluidas las creadas por Compose?

- A) `docker network ls`
- B) `docker compose network`
- C) `docker ps --networks`
- D) `docker inspect --net`

Pregunta 53

Dos proyectos Compose distintos, en carpetas distintas, usan ambos un servicio llamado `db`. ¿Entran en conflicto entre sí?

- A) Sí, siempre, porque los nombres de servicio son globales en todo el sistema
- B) No: Compose aísla cada proyecto en su propia red, así que los nombres no colisionan entre proyectos distintos
- C) Se fusionan automáticamente en un único proyecto compartido
- D) Docker lo prohíbe directamente al detectar el nombre repetido

Pregunta 54

En un entorno de producción, ¿por qué puede ser buena práctica limitar a qué redes pertenece cada servicio, en vez de ponerlos todos en la misma?

- A) No aporta ninguna ventaja real en la práctica
- B) Por seguridad: reduce la superficie de ataque si un servicio expuesto al exterior llega a verse comprometido
- C) Hace que Compose arranque más lento a propósito
- D) Compose no permite más de una red por proyecto, así que no hay elección

Bloque 7

depends_on y healthcheck

Pregunta 55

¿Para qué sirve la clave `depends_on:` en un servicio de Compose?

- A) Para indicar que ese servicio debe arrancarse después de otro(s) servicio(s) indicado(s)
- B) Para definir el puerto público de un servicio
- C) Para construir la imagen de otro servicio distinto
- D) Para eliminar automáticamente un servicio al terminar

Pregunta 56

Si `web` tiene `depends_on: [db]`, ¿garantiza eso que la base de datos esté completamente lista para aceptar conexiones antes de que arranque `web`?

- A) Sí, lo garantiza siempre por completo
- B) No necesariamente: por defecto solo garantiza el orden de arranque del contenedor, no que el servicio interno ya esté operativo
- C) `depends_on` no existe realmente en Compose
- D) Solo lo garantiza si el servicio usa `build:`

Pregunta 57

¿Qué estrategia se usa habitualmente en aplicaciones reales para que un servicio espere de verdad a que otro esté realmente disponible, más allá de `depends_on` básico?

- A) Ninguna: basta con confiar siempre en `depends_on`
- B) Añadir lógica de reintento/espera en la propia aplicación, o usar un `healthcheck` combinado con `depends_on`
- C) Arrancar todos los servicios a la vez, sin ningún orden definido
- D) Eliminar siempre `depends_on` del fichero

Pregunta 58

Dado:

```
services:  
  app:  
    depends_on:  
      - db  
  db:  
    image: postgres
```

¿En qué orden intentará Compose arrancar los contenedores?

- A) `app` y `db` exactamente al mismo tiempo
- B) Primero `db` , después `app`
- C) Primero `app` , después `db`
- D) El orden es aleatorio en cada ejecución

Pregunta 59

¿Puede un servicio depender de varios servicios a la vez mediante `depends_on` ?

- A) No, solo puede depender de uno
- B) Sí, se puede indicar una lista con varios nombres de servicio
- C) Solo de servicios que usen `build:`
- D) Solo si todos están en la misma red personalizada

Pregunta 60

Si el servicio indicado en `depends_on` no existe en el mismo `docker-compose.yml`, ¿qué ocurre?

- A) Se ignora silenciosamente y arranca igual
- B) Compose devuelve un error de configuración al intentar levantar los servicios
- C) Se crea automáticamente un servicio vacío con ese nombre
- D) No tiene ningún efecto sobre el arranque

Pregunta 61

¿Para qué sirve una clave `healthcheck:` en un servicio?

- A) Para definir el puerto público del servicio
- B) Para indicar cómo comprobar si el servicio está realmente "sano"/operativo, más allá de si el proceso ha arrancado
- C) Para construir la imagen a partir de un Dockerfile
- D) Para declarar variables de entorno adicionales

Pregunta 62

¿Es `depends_on` imprescindible para que dos servicios de la misma Compose puedan comunicarse por red?

- A) Sí, sin `depends_on` no pueden verse por la red interna
- B) No: `depends_on` solo afecta al orden de arranque; la comunicación por red funciona igual estén o no declaradas dependencias
- C) Solo es necesario si ambos usan volúmenes
- D) Solo es necesario en entornos de producción

Pregunta 63

Si eliminas `depends_on` de un `docker-compose.yml` con `app` y `db`, ¿qué riesgo concreto asumes?

- A) Ninguno: Compose siempre detecta y respeta el orden correcto igualmente
- B) Que `app` intente arrancar y conectar antes de que `db` esté lista, fallando la primera vez
- C) Que `db` deje de funcionar por completo
- D) Que `app` y `db` ya no puedan comunicarse nunca más

Bloque 8

Ciclo de vida: up / down / stop / start

Pregunta 64

¿Qué hace `docker compose up`?

- A) Solo construye las imágenes, sin arrancar ningún contenedor
- B) Crea (si hace falta) y arranca todos los servicios definidos en el `docker-compose.yml`
- C) Elimina todos los contenedores del proyecto
- D) Sube las imágenes del proyecto a Docker Hub

Pregunta 65

¿Qué diferencia hay entre `docker compose up` y `docker compose up -d` ?

- A) Ninguna diferencia relevante
- B) Sin `-d` se queda mostrando los logs en primer plano; con `-d` arranca en segundo plano y devuelve el control a la terminal
- C) `-d` elimina los contenedores nada más arrancar
- D) `-d` solo sirve para construir imágenes, no para arrancar

Pregunta 66

¿Qué hace `docker compose down` ?

- A) Solo pausa los contenedores del proyecto
- B) Para y elimina los contenedores y la red creados por ese proyecto Compose (y los volúmenes, solo si se indica explícitamente)
- C) Elimina todas las imágenes del sistema
- D) Reinicia por completo el servicio Docker del sistema

Pregunta 67

¿Qué opción de `docker compose up` fuerza a reconstruir las imágenes de los servicios que usan `build:` antes de arrancar?

- A) `--build`
- B) `--force-build`
- C) `--rebuild`
- D) `-r`

Pregunta 68

Añades un nuevo servicio al `docker-compose.yml` y ejecutas `docker compose up -d` de nuevo. ¿Qué ocurre con los servicios que no han cambiado?

- A) Se reinician todos sin excepción
- B) Compose normalmente los deja tal cual y solo crea/arranca lo que es nuevo o ha cambiado
- C) Se eliminan y se recrean desde cero obligatoriamente
- D) Da un error y hay que parar todo el stack a mano primero

Pregunta 69

¿Qué comando pararía los contenedores de un proyecto Compose sin eliminarlos?

- A) `docker compose stop`
- B) `docker compose pause-all`
- C) `docker compose halt`
- D) `docker compose kill-only`

Pregunta 70

¿Qué comando vuelve a arrancar servicios de un proyecto Compose que estaban parados con `stop`, sin recrearlos desde cero?

- A) `docker compose start`
- B) `docker compose up --new`
- C) `docker compose restart-all`
- D) `docker compose build`

Pregunta 71

Ejecutas `docker compose up -d` en una carpeta donde no hay ningún `docker-compose.yml` ni `compose.yaml`. ¿Qué ocurre?

- A) Se crea uno vacío automáticamente y arranca sin servicios
- B) Devuelve un error indicando que no encuentra el fichero de configuración
- C) Usa el último proyecto Compose ejecutado en el sistema, sea cual sea
- D) Levanta todos los contenedores descargados en el sistema

Pregunta 72

Si solo ejecutas `docker compose stop` (sin `down`), ¿qué pasa con la red y los volúmenes con nombre del proyecto?

- A) Se eliminan igualmente
- B) Se mantienen: solo se detienen los contenedores; es `down` quien elimina la red, y los volúmenes solo si se indica explícitamente
- C) Se convierten automáticamente en imágenes
- D) Dejan de existir de forma inmediata

Bloque 9

logs, ps, exec y run

Pregunta 73

¿Qué muestra `docker compose ps` ?

- A) Solo las imágenes descargadas en el sistema
- B) El estado de los contenedores de ese proyecto Compose (similar a `docker ps` , pero filtrado al proyecto)
- C) Los logs combinados de todos los servicios
- D) El contenido de los volúmenes del proyecto

Pregunta 74

¿Qué hace `docker compose logs` (sin argumentos)?

- A) Muestra los logs combinados de todos los servicios del proyecto
- B) Elimina los logs antiguos acumulados
- C) Reconstruye las imágenes de los servicios
- D) Lista los volúmenes existentes

Pregunta 75

¿Cómo verías únicamente los logs de un servicio concreto, por ejemplo `db`, dentro de un proyecto Compose?

- A) `docker compose logs db`
- B) `docker logs --compose db`
- C) `docker compose ps db`
- D) No es posible filtrar los logs por servicio

Pregunta 76

¿Qué hace la opción `-f` en `docker compose logs -f` ?

- A) Fuerza a borrar todos los logs acumulados
- B) Sigue mostrando los logs en tiempo real, igual que `tail -f`
- C) Filtra para mostrar solo mensajes de error
- D) Formatea la salida como JSON

Pregunta 77

¿Cómo abrirías una sesión interactiva de terminal dentro de un servicio llamado `app` que ya está en marcha, gestionado por Compose?

- A) `docker compose exec app bash` (o `sh` si la imagen no tiene bash)
- B) `docker compose run app`
- C) `docker compose enter app`
- D) `docker exec app --compose`

Pregunta 78

¿Qué diferencia hay entre `docker compose exec` y `docker compose run` para un mismo servicio?

- A) Son exactamente equivalentes en todos los casos
- B) `exec` ejecuta un comando en un contenedor de ese servicio que YA está en marcha; `run` crea un contenedor nuevo para ejecutar algo puntual
- C) `run` solo puede usarse con servicios de base de datos
- D) `exec` siempre crea un contenedor completamente nuevo

Pregunta 79

Si un servicio `web` no arranca correctamente en Compose, ¿cuál sería un primer paso razonable para investigar por qué?

- A) Ejecutar `docker compose down` directamente sin mirar nada más
- B) Revisar sus logs con `docker compose logs web`
- C) Ejecutar `docker compose build --no-cache` sin comprobar antes qué falla
- D) Borrar todo el proyecto y empezar de nuevo desde cero

Pregunta 80

¿Qué comando mostraría en tiempo real los logs combinados de TODOS los servicios de un proyecto Compose?

- A) `docker compose logs -f`
- B) `docker compose ps -f`
- C) `docker compose build -f`
- D) `docker compose down -f`

Pregunta 81

El equipo necesita lanzar un comando puntual de migración de base de datos usando la misma imagen que el servicio `app`, sin tocar el contenedor que ya está en marcha en producción. ¿Qué comando de Compose encaja mejor?

- A) `docker compose exec app ...`, reutilizando siempre el contenedor ya activo
- B) `docker compose run app ...`, que crea un contenedor nuevo y aislado de ese servicio para la tarea puntual
- C) `docker compose down` seguido de `up` de nuevo
- D) No es posible lanzar tareas puntuales con Compose

Bloque 10

Buenas prácticas y referencias

Pregunta 82

¿Por qué conviene usar un volumen con nombre para los datos de la base de datos en un stack típico "app + base de datos"?

- A) Para que la base de datos ocupe menos espacio en disco
- B) Para que los datos sobrevivan a `docker compose down` y a la recreación de los contenedores
- C) No es necesario: Compose guarda siempre los datos dentro de la propia imagen
- D) Para acelerar la velocidad de la red interna

Pregunta 83

¿Por qué no es buena idea escribir contraseñas reales directamente en un `docker-compose.yml` que se sube a un repositorio público?

- A) No hay ningún problema: Compose las oculta automáticamente al mostrarlo
- B) Porque cualquiera con acceso al repositorio podría verlas; es preferible usar variables de entorno o un `.env` no versionado
- C) Docker Compose no permite usar contraseñas en ningún caso
- D) Las contraseñas en texto plano no afectan a la seguridad real del sistema

Pregunta 84

¿Qué ventaja tiene mantener un `docker-compose.yml` frente a documentar una lista de comandos `docker run` sueltos en el README?

- A) Ninguna relevante, da exactamente igual
- B) El fichero es directamente ejecutable (`docker compose up`) y actúa como documentación viva, siempre sincronizada con lo que realmente se ejecuta
- C) `docker run` siempre arranca más rápido que Compose
- D) Compose no permite reproducir exactamente el mismo entorno entre máquinas

Pregunta 85

En un stack "app + base de datos", ¿cuál es la forma de comunicación recomendada entre ambos servicios?

- A) La app accede a la base de datos por su IP pública de internet
- B) La app se conecta a la base de datos usando el nombre del servicio como host, dentro de la red interna creada por Compose
- C) No deberían poder comunicarse nunca entre sí
- D) Solo mediante ficheros compartidos, nunca por red

Pregunta 86

Se actualiza la imagen base de un servicio de base de datos y se reconstruye el stack, sin haber hecho copia de seguridad del volumen de datos antes. ¿Qué riesgo se asume?

- A) Ninguno: es siempre completamente seguro
- B) Riesgo de incompatibilidad entre versiones de datos; conviene revisar la documentación de la imagen y hacer copia de seguridad antes de actualizar
- C) El volumen se actualiza solo, sin ningún riesgo real
- D) Los datos desaparecen siempre de forma automática al cambiar de versión

Pregunta 87

¿Por qué es buena práctica fijar versiones concretas (p. ej. `postgres:16`) también en los servicios de un `docker-compose.yml`, en vez de usar `latest`?

- A) Para evitar cambios de versión inesperados al reconstruir el stack en el futuro
- B) Porque `latest` directamente no funciona dentro de Compose
- C) Porque Compose lo exige de forma obligatoria
- D) No hay ninguna razón práctica real para hacerlo

Pregunta 88

¿Qué relación tiene todo lo visto sobre Compose con el "despliegue de aplicaciones web" del módulo?

- A) Ninguna, es un tema completamente aislado del resto del módulo
- B) Compose (y Docker en general) son herramientas reales para empaquetar y desplegar de forma reproducible una aplicación web junto con sus dependencias
- C) Solo sirve para el entorno de desarrollo local, nunca para producción
- D) Sustituye por completo la necesidad de tener un servidor

Pregunta 89

¿Qué papel juega un `.dockerignore` bien configurado en un proyecto Compose que usa `build:` ?

- A) Ninguno: Compose no usa contexto de build en ningún caso
- B) Evita incluir ficheros innecesarios o sensibles (como un `.env` con credenciales) dentro de la imagen construida
- C) Hace que los contenedores ya creados arranquen más rápido
- D) Es obligatorio para que `depends_on` funcione correctamente

Pregunta 90

¿Qué suele documentarse primero en el README de un proyecto que usa Docker Compose, para que cualquiera pueda levantarlo?

- A) El código fuente completo de la aplicación
- B) El comando `docker compose up` y los requisitos previos (como variables de entorno necesarias)
- C) La contraseña real usada en producción
- D) El historial completo de commits del repositorio

Bloque 11

Preguntas de escenario

Pregunta 91

Un stack con `app` y `db` falla al arrancar: `app` da un error de conexión a la base de datos justo al inicio, aunque tiene `depends_on: [db]`. ¿Cuál es la explicación más probable?

- A) `depends_on` está mal escrito en el YAML
- B) El contenedor de `db` ya había arrancado, pero el motor de base de datos dentro de él todavía no estaba listo para aceptar conexiones cuando `app` lo intentó
- C) Docker Compose no soporta bases de datos en absoluto
- D) Es imposible que esto llegue a ocurrir con Compose

Pregunta 92

Tras `docker compose down` (sin `-v`) y volver a hacer `docker compose up`, los datos de la base de datos siguen intactos. ¿Por qué?

- A) Es pura casualidad
- B) Porque los datos estaban en un volumen con nombre, que `down` no elimina por defecto
- C) Porque `down` nunca borra absolutamente nada
- D) Porque la base de datos los reenvía automáticamente a Docker Hub

Pregunta 93

Dos desarrolladores levantan el mismo proyecto Compose en fechas distintas y dicen "en mi máquina se comporta distinto"; uno usaba `postgres:latest` y descargó una versión más reciente que el otro. ¿Qué cambio evitaría este problema?

- A) Ninguno: es imposible de evitar con Docker
- B) Fijar una versión concreta, por ejemplo `postgres:16`, en vez de `latest`
- C) Eliminar directamente el servicio de base de datos
- D) Sustituir `image:` por `build:` sin más

Pregunta 94

Un servicio `web` necesita esperar a que `api` esté realmente disponible (no solo arrancado) antes de empezar a servir peticiones. ¿Qué mecanismo de Compose encaja mejor con esta necesidad?

- A) `ports:`
- B) Un `healthcheck:` en `api` combinado con una condición en `depends_on` de `web`
- C) `volumes:`
- D) `networks:` personalizadas, sin nada más

Pregunta 95

Un equipo quiere que la base de datos de su stack NO sea accesible desde fuera de la máquina en producción, pero sí poder conectarse puntualmente desde su portátil para depurar un problema concreto. ¿Qué estrategia encaja mejor?

- A) No declarar `ports:` en la configuración de producción, pero sí publicarlo temporalmente solo cuando haga falta depurar (p. ej. en una configuración de desarrollo aparte)
- B) Publicar siempre el puerto de la base de datos en todos los entornos por comodidad
- C) Es imposible depurar sin publicar el puerto de forma permanente
- D) Cambiar el nombre del servicio soluciona el problema de acceso

Pregunta 96

Tras añadir un tercer servicio (`cache`) al `docker-compose.yml` y ejecutar `docker compose up -d` , `web` y `db` no se reinician. ¿Es esto un problema?

- A) Sí, siempre hay que reiniciar todos los servicios al añadir uno nuevo
- B) No necesariamente: Compose solo crea/arranca lo necesario, dejando en marcha los servicios que no han cambiado
- C) Indica que Compose está roto y hay que reinstalarlo
- D) Significa que `cache` en realidad no se ha llegado a crear

Pregunta 97

Un `docker-compose.yml` de producción publica con `ports:` tanto la app web como la base de datos directamente hacia internet. ¿Qué recomendación de seguridad aplicarías?

- A) No publicar la base de datos hacia internet; exponer solo lo estrictamente necesario y dejar que el resto se comuniquen por la red interna de Compose
- B) Está todo correcto: cuantos más puertos abiertos, mejor
- C) Eliminar `depends_on` para mejorar la seguridad
- D) Usar `latest` en todas las imágenes para estar siempre actualizado

Pregunta 98

Un alumno sube accidentalmente un `.env` con contraseñas reales de producción a un repositorio público que usa Compose. Además, ese `.env` ya llevaba varios commits confirmado en el historial. ¿Basta con borrarlo ahora y añadirlo a `.gitignore`?

- A) Sí, desaparece automáticamente del historial de git al borrarlo
- B) No del todo: aunque se borre y se ignore a partir de ahora, las versiones anteriores del fichero siguen en el historial de git, visibles para quien lo clone
- C) Sí, porque `.gitignore` limpia también los commits anteriores
- D) No tiene ninguna importancia real una vez pasado el susto

Pregunta 99

El equipo decide documentar en el `docker-compose.yml` de desarrollo un bind mount del código fuente (`. : /app`) para el servicio `app` , pero en producción prefieren copiar el código con `COPY` dentro de la imagen. ¿Por qué tiene sentido esta diferencia entre entornos?

- A) No tiene ningún sentido, deberían configurarse siempre igual
- B) En desarrollo interesa ver los cambios de código al instante sin reconstruir; en producción interesa una imagen inmutable y autocontenida, sin depender de ficheros externos del host
- C) Un bind mount nunca funciona en Linux, solo en desarrollo con Windows
- D) `COPY` y bind mount hacen exactamente lo mismo en cualquier entorno

Pregunta 100

Resume el flujo típico para levantar un stack app + base de datos con Compose, comprobar que funciona y pararlo dejando los datos a salvo:

- A) `docker build` → `docker run` → `docker rmi`
- B) `docker compose up -d` → `docker compose logs` / `docker compose ps` → `docker compose down` (sin `-v`, para conservar los volúmenes)
- C) `docker pull` → `docker push` → `docker login`
- D) `docker compose down -v` → `docker compose up` → `docker compose build`

¡Fin del bloque 1.2 Docker — suerte!